PROJECT-BASED LEARNING

PHASE-I EVALUATION

Satellite Health Monitoring and Collision Avoidance

Team ID: DSCPP-III-2026-T004

Department of Computer Science & Engineering
Graphic Era Deemed to be University, Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T004
Semester: 3rd
Section: E, ML2, DS1, H
Domain: DS/CPP

Team Members

1. Chhavit (Lead) - 2510011425
2. Harsh Jain - 2510370400
3. Satyam Kumar - 2510380092
4. Sudhanshu Ojha - 2510011648

Mentor

Mr. Utkarsh Pant

Interactions completed: _____

Problem Statement & Motivation

Problem Statement

Space is becoming dangerously crowded with debris. Currently, satellite collision avoidance and health management rely on slow, manual human-in-the-loop Earth-based radar and command systems. Delays in human intervention for software glitches or space junk dodging can result in catastrophic collisions.

Motivation

- Manual tracking and updating take too much time and effort.
- Exponential increase in satellites makes human-in-the-loop methods unscalable.

Target Users / Stakeholders

- Space Agencies (e.g., ISRO, NASA)
- Private Aerospace Corporations
- Expected Benefit: Real-time, automated collision avoidance and reduced human operational overhead.

Objectives

Project Objectives

1. Develop a C++ simulation for autonomous satellite navigation and health monitoring.
2. Map 3D local space using an AVL Tree to optimize debris detection speeds.
3. Prioritize imminent collision threats using a Min-Heap data structure.
4. Integrate OOP concepts to allow specialized satellite types to auto-repair software glitches.
5. Stress-test the system via a random "Chaos Engine" and log all actions.

Key Objective: Build a functional C++ simulator where satellites detect, prioritize, and dodge debris autonomously without human input.

Proposed Solution

Proposed Approach

1. **Input:** Chaos Engine throws random debris and system errors.
2. **Processing:** Grid Tracker (AVL Tree) maps local space sectors.
3. **Logic:** Threat Radar (Min-Heap) targets nearest junk; Flight Computer dodges.
4. **Storage:** Circular Queues track rolling battery health history.
5. **Output:** Live terminal updates and blackbox.txt log generation.

Key Features

- Automated 3D Debris Avoidance
- Object-Oriented Self-Healing
- AVL Tree Space Sectoring
- Min-Heap Threat Prioritization
- Comprehensive Flight Logging

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C/C++	AVL Trees (Grid mapping), Min-Heaps (Priority Queues), Circular Queues (Battery tracking).	Threat Radar & Grid Tracker
OOPs with C++	Classes/Objects, Inheritance (Satellite Types), Polymorphism (Self-healing), Encapsulation (Fuel levels).	Flight Computer & Entities

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow

Major Components

- Chaos Engine (Hazard Generator)
- Grid Tracker (AVL Tree) & Radar (Heap)
- OOP Flight Computer & Diagnostics

Data / Control Flow

- Engine generates localized debris objects.
- Radar sorts hazards by immediate proximity.
- Computer fires thrusters; logs event to file.

Technology Stack & Methodology

Technology Stack

- Programming: C++
- Database/Storage: File Handling (.txt logs)
- Tools / Framework: GCC, C++ Standard Library
- IDE: VS Code / Code::Blocks
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis (OOP vs DSA limits)
2. System Design (Architecture diagram)
3. Implementation (Iterative C++ coding)
4. Testing (Chaos Engine stress testing)
5. Evaluation (Log analysis)

Initial Progress & Team Contribution

Progress Achieved

- Drafted OOP base and derived classes.
- Identified core DSA integration points.
- Designed simulation flow and math logic.
- Initialized GitHub repository.
- Mapped out the Blackbox logging structure.

Individual Contribution

Member	Role	Contribution
Chhavit	Team Lead/OOP	Flight Computer & OOP Architecture
Harsh Jain	Developer	Chaos Engine & Random Events
Satyam Kumar	DSA Dev	AVL Trees & 3D Grid Mapping
Sudhanshu Ojha	DSA Dev/Docs	Min-Heap Radar & Log Handling

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal, OOP & DSA Design
2. Phase-II: Module Coding & Integration
3. Phase-III: Testing, Debugging & Final Log

Expected Outcomes

- Terminal-based C++ visual simulation.
- Permanent blackbox mission history.

References

1. C++ documentation for Inheritance/Polymorphism.
2. Algorithms for self-balancing AVL Trees.
3. Priority Queue designs using Min-Heaps.
4. Basic 3D math for coordinate distances.

Thank You!
Any Questions ? & Suggestions